

BOS-OMEGA

Governance Layer

Specification of the policy, control, and audit machinery that governs every LLM call inside the BOS-OMEGA orchestration platform.

Document scope

This document covers the components that decide whether a request runs, which model handles it, what counts as an acceptable answer, and what gets recorded. It does not cover the model adapters themselves, transport-level security, or product UX.

Codebase	BOS-OMEGA (Express 5 + React/Vite + PostgreSQL, pnpm monorepo)
Governance code path	artifacts/api-server/src/bos/
Persistence	PostgreSQL via Drizzle ORM (lib/db/src/schema/*)
Generated	2026-04-26
Document version	1.0

1. Executive overview

BOS-OMEGA does not let an LLM decide whether to act. Every request flows through a fixed, auditable pipeline of governance checkpoints before, during, and after the model call. Each checkpoint is a small, single-purpose module with a clear input and output contract; the pipeline orchestrator (pipeline.ts) is the only place that wires them together.

The layer is built around six guarantees:

- Every request is screened by an input gate before any model is selected.
- Every executable decision passes through a probabilistic Tri-State engine (GO / HOLD / ABORT) whose vector and evidence signals are persisted.
- Model selection is deterministic, capability-based, and respects per-provider circuit breakers.
- Every model output is validated against a schema and safety rules; failed outputs are repaired, not silently returned.
- Long-lived behavioural rules live in a Memory layer with an authority hierarchy — Canon is system law.
- Every state transition is appended to an immutable audit log, keyed by task ID, with structured metadata.

This document is the canonical reference for what each checkpoint does and what it persists. Implementation files are cited inline.

2. Pipeline topology

Every task entering the orchestrator follows the same ordered sequence. The orchestrator is implemented in `artifacts/api-server/src/bos/pipeline.ts:runBosPipeline` and is the single public entry point.

2.1 Stages, in order

- Receive — task ID minted, attachments loaded, `TASK_RECEIVED` audited.
- Input gate — pattern-based safety screen + sanitisation + intent detection.
- Task classifier — determines `task_type` (legal, code, math, research, etc.).
- Mode selector — picks normal / `series_pass` / `boil_the_ocean` from input cues.
- Model router — capability-matrix scoring across enabled, healthy models.
- Tri-State evaluation — qubit-inspired vector, persisted to `tri_state_decisions`.
- Execution — single, parallel, series-pass, or boil-the-ocean engine.
- Validation + repair — schema, safety, instruction-fit, completeness.
- Persist final task state and emit `TASK_COMPLETED` / `HELD` / `ABORTED`.

2.2 Terminal states

Three terminal states are possible for a task; nothing else is exposed to callers:

GO / COMPLETED	Tri-state collapsed to GO and validation passed (or repair recovered it). Final answer is returned.
HOLD / HELD	Either input gate or Tri-State collapsed to HOLD because of missing inputs, low confidence, or weak provider availability. Caller is told what is missing.
ABORT / ABORTED	Either the input gate matched a hard-block pattern, or the Tri-State ABORT amplitude crossed 0.70. No model is called for safety aborts.

3. Input gate

File: `artifacts/api-server/src/bos/inputGate.ts`. The gate runs before any model is selected and is the platform's first line of defence.

3.1 Responsibilities

- Sanitise the raw input — strip control bytes, neutralise `<script>` blocks and javascript: URIs, trim, hard-cap at 32,000 characters.
- Match against a hard-block pattern set; matched requests collapse to ABORT immediately with `risk_level = high`.
- Detect missing-context shapes; very short or vague prompts

collapse to HOLD with missing_info populated.

- Classify intent (code, legal, math, research, summarization, extraction, planning, creative, safety_review, general) for downstream routing.
- Assign a coarse risk_level (none / low / medium / high) used as a Tri-State signal.

3.2 Hard-block pattern categories

These are intentionally narrow and behavioural — not topic bans. They target unambiguous misuse:

- Active intrusion: hack/crack/exploit/bypass paired with system / server / database / account.
- Malware authorship: create/generate/make/build paired with malware / virus / ransomware / trojan / spyware.
- Synthesis of controlled substances or explosives.
- Targeted instructions for harming a specific person.
- Credential phishing and social-engineering instructions.
- Any reference to child sexual exploitation material.

Sample matched-pattern shape (TypeScript)

```
/\b(hack|crack|exploit|bypass)\b.*\b(system|server|database|account)\b/i,  
/\b(create|generate|make|build)\b.*\b(malware|virus|ransomware|trojan|spyware)\b/i,  
/child.*(sexual|pornograph|exploit)/i,
```

3.3 Output contract

state	GO HOLD ABORT
sanitized_input	Cleaned text used downstream — the raw input is never forwarded to a model
intent	One of the classified intents above
risk_level	none low medium high
missing_info	Array of named missing inputs (e.g. ['sufficient_context'])
reason	Human-readable explanation when state is HOLD or ABORT

4. Tri-State decision engine

File: artifacts/api-server/src/bos/triState.ts. Persisted in tri_state_decisions (lib/db/src/schema/triStateDecisions.ts).

The engine is qubit-inspired but not literal quantum computing. GO, HOLD, and ABORT exist as weighted decision amplitudes (a probability vector) before being collapsed to a single runtime state by deterministic threshold rules. The vector is advisory; only the collapsed state controls execution.

4.1 Initial state and signal application

Each task starts at the neutral vector { go: 0.33, hold: 0.34, abort: 0.33 }. Evidence signals are gathered from the input gate, classifier, router, and (when available) earlier validation results. Each signal carries an explicit impact on each amplitude; impacts are summed, negative components are clamped to zero, and the vector is renormalised after every signal.

4.2 Evidence signal categories

safety	illegal_instruction (+0.80 abort), unauthorized_action (+0.60 abort), unsafe_request, safe_request
risk	medium_risk_intent, low_risk_intent, high_stakes_domain (legal, medical, financial, research, code)
completeness	missing_required_inputs (+0.45 hold), required_inputs_present, validation_passed, validation_failed
intent_clarity	clear_user_intent (≥ 0.75), ambiguous_user_intent (≤ 0.40), ambiguity_detected
confidence	high_confidence (≥ 0.75), low_confidence (≤ 0.40)
source_quality	strong_source_quality, weak_source_quality, unsupported_factual_claim
tool_availability	no_provider_available + no_fallback (+0.40 hold), primary_provider_available, providers_available_with_fallback

4.3 Collapse rules

After all signals are applied, the final vector is collapsed using these rules in order:

```
if (vector.abort >= 0.70)           -> ABORT
if (vector.hold >= 0.45 && hasMissingOrErrors) -> HOLD
if (vector.go >= 0.60 && validationPassed)   -> GO
otherwise                                -> HOLD (default)
```

ABORT is intentionally hard to reach — it requires roughly 70 percent of the amplitude after normalisation, which only happens when one or more strong safety signals fire. HOLD is the safe default whenever no other rule resolves cleanly. GO is the only state that allows an LLM to be called.

4.4 What gets persisted per decision

task_id	Loose link to the task (decision is created before the task row exists)
go_score / hold_score / abort_score	Final amplitudes after normalisation
evidence_signals	JSON array of every signal that fired, with impact deltas and human-readable description
collapse_reason	Which threshold rule fired (or why no rule fired)
final_state	GO HOLD ABORT — the only field that controls execution
confidence_score	Dominance of the chosen amplitude over the runner-up — bounded to [0,1]
created_at	Timestamp of the decision

5. Mode selector

File: artifacts/api-server/src/bos/modeSelector.ts. Determines how much compute is spent on a request.

5.1 Modes

normal

Single best-fit model. Default for short, low-stakes tasks.

series_pass

Sequential refinement chain across N models — each pass critiques and improves the previous output.

boil_the_ocean

Parallel agent fan-out across multiple models, then synthesis, then adversarial review. Reserved for high-stakes or 'final-version' work.

auto

Mode selector decides based on intent keywords, task type, and input length.

5.2 Auto-selection signals

- Boil-the-ocean keywords: 'boil the ocean', 'exhaustive', 'final version', 'no ambiguity', '10/10', 'comprehensive', 'production ready', 'ship it', 'enterprise', 'bulletproof'.
- Series-pass keywords: 'error check', 'improve', 'refine', 'review', 'critique', 'validate', 'polish', 'iterate', 'perfect this'.
- High-stakes task type (legal / research / planning / code) with input > 200 chars promotes to boil-the-ocean.
- Complex task type with input > 100 chars promotes to series-pass.
- Inputs longer than 500 chars default to series-pass.
- Otherwise: normal.

Explicit user override always wins. The selector returns a mode plus the reason and confidence, both audited via `MODE_SELECTED`.

6. Model router and circuit breaker

6.1 Capability matrix (model router)

File: artifacts/api-server/src/bos/modelRouter.ts. The router scores every enabled model in every enabled provider against a fixed capability requirement set per task type.

legal	-> [reasoning, legal, long_context]
code	-> [coding, reasoning, structured_output]
math	-> [reasoning, structured_output]
research	-> [research, long_context, reasoning]
summarization	-> [cheap, fast, long_context]
extraction	-> [structured_output, fast]
planning	-> [reasoning, structured_output]
creative	-> [fast, cheap]
safety_review	-> [safety, reasoning]
general	-> [reasoning, fast]

Final score per model is a weighted sum:

score = capability_match	* 3.0
+ reliability_score	* 2.0
+ context_fit	* 1.5
+ latency_score	* 1.0
+ cost_score	* 0.5
+ provider_health	* 2.0

Models whose provider health is OPEN_CIRCUIT are filtered out before scoring. Context fit drops to zero when estimated tokens exceed 90 percent of the model's window.

6.2 Circuit breaker

File: artifacts/api-server/src/bos/circuitBreaker.ts. State persisted in provider_health (lib/db/src/schema/providerHealth.ts).

Provider health states

HEALTHY	Default. Eligible for routing. Latency tracked as an EWMA (avg_latency_ms = 0.8 * prev + 0.2 * latency).
DEGRADED	Triggered after 3 consecutive failures from HEALTHY, or by 3+ schema failures. Penalised in routing score; not blocked.
OPEN_CIRCUIT	Triggered by an auth failure, or by 5+ failures within a 10-minute window. Excluded from routing entirely.
RECOVERY_TEST	Half-open probe. A success returns the provider to HEALTHY.

Recorded events

- recordSuccess updates last_success, EWMA latency, and may auto-recover from RECOVERY_TEST.
- If a successful call exceeds 5,000 ms, the provider is auto-marked DEGRADED.
- recordFailure increments failure counters and applies the rules above; CIRCUIT_BREAKER_OPENED is audited.
- Schema failures are tracked separately and have their own threshold (3) for DEGRADED.

7. Output validation and repair

7.1 Validation engine

File: artifacts/api-server/src/bos/validationEngine.ts. Persisted in validation_results.

Every model output is validated on four independent axes before the answer is returned to the caller. The output must parse to a JSON object that matches the BosOutput schema.

schema_pass	JSON parses, state " {GO, HOLD, ABORT}, answer is a string, task_type is a string.
safety_pass	Output text does not match unsafe-output patterns (e.g. step-by-step bomb, hack into ..., gain unauthorized access).
instruction_pass	Parsed output has both state and answer; answer is at least 5 characters.
completeness_pass	answer "e 10 chars and recommended_next_action is present. A confidence_score is computed from the four flags (0.3 + 0.3 + 0.2 + 0.2), then attenuated when the model self-reports many uncertainties or any failure_modes. validation.passed is true only when all four flags pass and confidence "e 0.5.

7.2 Repair engine

File: artifacts/api-server/src/bos/repairEngine.ts. The repair engine is intentionally conservative — it patches structure, not substance.

- Schema repair: extracts the largest JSON object from the output; missing fields are filled with safe defaults (state=GO, task_type=general, empty arrays for assumptions / uncertainties / missing_inputs / failure_modes, recommended_next_action='Review the answer above'). If no JSON is found, the raw text becomes the answer field of a synthesised BosOutput.
- Completeness repair: ensures recommended_next_action exists and that answer is not trivially short.
- Drift removal: rewrites self-references like 'I'm Claude' / 'I'm GPT' / 'As an AI assistant' to 'BOS-OMEGA', preventing model branding from leaking into the answer.

REPAIR_APPLIED is audited. If validation still fails after repair, the orchestrator may downgrade the task's Tri-State to HOLD via the validation_failed signal on a subsequent evaluation.

8. Memory layer

Files: artifacts/api-server/src/bos/memoryEngine.ts, lib/db/src/schema/memory.ts, artifacts/bos-omega/src/pages/MemoryManager.tsx.

Memory items are durable, layered, ordered by an authority field, and surfaced as context blocks injected ahead of every prompt.

8.1 Layers (highest authority first)

canon	System law. Highest authority. Items cannot be deleted without an explicit type-the-title confirmation in the admin UI. Top 10 canon items by <code>authority_level</code> are prepended to every prompt as the [CANON CONTEXT] block.
patches	Targeted overrides — small, surgical adjustments to canon behaviour.
continuity	Long-running facts about the user / domain / project that should persist across sessions.
logs	Structured operational notes generated by the system.
scratchpad	Short-lived working notes. Top 5 most recently updated are surfaced as the [SCRATCHPAD] block.

8.2 Authority and admin powers

Every item carries an `authority_level` (1–10). Within a layer, higher authority wins. Across layers, canon outranks every other layer regardless of authority. Authority is editable.

CRUD endpoints: GET / POST / PATCH `/api/memory` and DELETE `/api/memory/:id`. All endpoints sit behind the `global requireAuth` middleware. The admin UI requires the operator to type the rule's exact title before the destructive button enables for canon-layer deletes.

8.3 Context block format

```
=== CANON CONTEXT ===
[CANON:<title>] <content>
[CANON:<title>] <content>
...

=== SCRATCHPAD ===
[SCRATCHPAD:<title>] <content>
...
```

9. Audit log

File: `artifacts/api-server/src/bos/auditEngine.ts`. Persisted in `audit_logs` (`lib/db/src/schema/audit.ts`).

Every state transition the orchestrator makes is appended to the audit log with a typed `event_type`, a human-readable message, and a structured metadata JSON blob keyed by `task_id`. Audit writes never throw — failures are logged but never block the pipeline.

9.1 Event types

- Lifecycle: `TASK_RECEIVED`, `INPUT_GATE_RESULT`, `TASK_CLASSIFIED`, `TRI_STATE_EVALUATED`, `MODEL_SELECTED`, `TASK_COMPLETED`, `TASK_ABORTED`,

TASK_HELD, MODE_SELECTED.

- LLM call: LLM_CALL_STARTED, LLM_CALL_COMPLETED, LLM_CALL_FAILED, FALLBACK_TRIGGERED.
- Output handling: VALIDATION_COMPLETED, REPAIR_APPLIED.
- Parallelism: PARALLEL_EXECUTION_STARTED, PARALLEL_EXECUTION_COMPLETED, MERGE_COMPLETED.
- Boil-the-ocean: BTO_STARTED, BTO_AGENTS_DISPATCHED, BTO_AGENTS_COMPLETED, BTO_SYNTHESIS_STARTED, BTO_SYNTHESIS_COMPLETED, BTO_ADVERSARIAL_COMPLETED, BTO_ABORTED, BTO_COMPLETED.
- Series pass: SERIES_PASS_STARTED, SERIES_PASS_STEP, SERIES_PASS_ABORTED, SERIES_PASS_COMPLETED.
- Provider health: CIRCUIT_BREAKER_OPENED, CIRCUIT_BREAKER_CLOSED.
- Attachments: ATTACHMENT_NOTES (per-attachment processing notes — extraction skipped, transcription unavailable, etc.).
- Provider keys: PROVIDER_KEY_UPDATED, PROVIDER_KEY_CLEARED.

9.2 Per-event metadata examples

TRI_STATE_EVALUATED	{ reason, go, hold, abort, confidence, signals: <count>, decision_id }
MODE_SELECTED	{ reason, confidence, requested }
MODEL_SELECTED	{ count }
TASK_RECEIVED	{ mode, input_length, attachments, attachment_images, attachment_context_chars }
ATTACHMENT_NOTES	{ notes: string[] } — honest skipped status when transcription is unavailable

10. Attachments and multimodal context

Files: artifacts/api-server/src/lib/uploads/*, artifacts/api-server/src/routes/uploads.ts, artifacts/api-server/src/bos/providerBridge.ts.

Attachments are first-class governance objects. They are loaded at the start of the pipeline so that their existence, count, and processing notes are auditable before any model is selected.

10.1 Storage and extraction

- Local-disk storage with SHA-256 deduplication.
- Text extraction for PDF (pdf-parse), DOCX (mammoth), and a wide range of utf-8 text/code/csv/json formats.
- Image metadata via sharp; thumbnails generated for the UI.
- Video frames extracted via ffmpeg; audio stripped and routed to transcription.

- Audio and video transcription via OpenAI Whisper when an OpenAI key is configured. When not configured, the attachment record is marked `extraction_status=skipped` and an `ATTACHMENT_NOTES` entry records the honest reason. Skipped transcription never produces a fabricated transcript.

10.2 Vision routing

Image bytes are forwarded to a model only when its `capability_tags` include the 'multimodal' marker. Both `executionEngine.buildOptions` and `providerBridge.callProviderDirect` enforce this gate, and the gate covers all five adapter branches: OpenAI, Anthropic, Gemini, Ollama, and the generic OpenAI-compatible adapter. Models without the tag receive the extracted text context block but no image bytes — never silently dropped without a record.

10.3 What reaches the model

attachment_context

Concatenated extracted text from every attachment, prepended to the prompt as a structured block.

attachment_images

`VisionImage[]` (id + mime + base64 bytes) — passed only to multimodal-tagged models.

task linkage

Attachment rows are updated with `task_id` immediately after the task row is minted, so audit and storage stay joinable.

11. Authentication and authorisation

Files: `artifacts/api-server/src/routes/auth.ts`, `artifacts/api-server/src/routes/index.ts`.

BOS-OMEGA uses a single-admin authentication model. The admin password is loaded from the `ADMIN_PASSWORD` environment variable (or hashed equivalent `ADMIN_PASSWORD_HASH`). When neither is present, the server generates a one-time random password at boot, prints it to the log once, and refuses to print it again — operators are pushed to set a stable value.

11.1 Session contract

- `POST /api/auth/login` validates the password and issues an HMAC-SHA256-signed `bos_session` cookie.
- All `/api/*` routes other than `/auth/login` and `/health` sit behind the `requireAuth` middleware.
- Cookies carry `{ iat, exp, sid }`; signature uses

SESSION_SECRET. Tampered cookies are rejected.

11.2 Defence in depth

- helmet for HTTP hardening and a strict Content-Security-Policy.
- Per-route rate limiting via express-rate-limit; an expensiveLimiter is applied to /api/uploads.
- Server-side body limits to bound memory.
- SSRF protection via safeFetch for any outbound URL given to the server.
- Provider API keys encrypted at rest with AES-256-GCM.
- Sanitised error messages in production — internal stack traces never leak through HTTP responses.

12. Execution-mode governance impact

12.1 Normal

Single best-fit model. The simplest path: validate, repair, return. One LLM call per task (plus optional fallback).

12.2 Series pass

File: artifacts/api-server/src/bos/seriesPassEngine.ts. Up to 7 models are fetched. Each pass takes the previous output, audits a SERIES_PASS_STEP entry, and runs a critique-and-improve prompt. If any pass collapses to ABORT the chain stops and SERIES_PASS_ABORTED is audited. Attachment context and images are threaded through every pass via providerBridge.callProviderDirect.

12.3 Boil the ocean

File: artifacts/api-server/src/bos/boilTheOceanEngine.ts. Up to 10 models are fetched and fanned out as agents in parallel; the orchestrator then runs a synthesis pass and an adversarial review pass. Three callsites in this engine — agent fan-out, synthesis, and adversarial review — all forward the attachment bundle. Each phase emits its own audit event (BTO_AGENTS_DISPATCHED, BTO_AGENTS_COMPLETED, BTO_SYNTHESIS_*, BTO_ADVERSARIAL_COMPLETED, BTO_COMPLETED).

12.4 Governance invariants across all modes

- Tri-State and validation are evaluated before any answer is returned, regardless of mode.
- Every model attempt produces a `model_attempts` row with provider, model, latency, token counts, raw response, and parallel grouping.
- Failures from any single model never propagate as a successful answer — the worst case is HOLD with a recorded reason.

13. Persisted governance tables

Tables in `lib/db/src/schema/` that exist specifically to make governance auditable after the fact:

audit_logs	<code>id</code> , <code>task_id</code> , <code>event_type</code> , <code>message</code> , <code>metadata</code> JSONB, <code>created_at</code> — append-only event stream.
tri_state_decisions	<code>id</code> , <code>task_id</code> , <code>go_score</code> , <code>hold_score</code> , <code>abort_score</code> , <code>evidence_signals</code> (JSON), <code>collapse_reason</code> , <code>final_state</code> , <code>confidence_score</code> , <code>created_at</code> .
validation_results	<code>id</code> , <code>task_id</code> , <code>attempt_id</code> , <code>schema_pass</code> , <code>safety_pass</code> , <code>instruction_pass</code> , <code>completeness_pass</code> , <code>confidence_score</code> , <code>notes</code> , <code>created_at</code> .
model_attempts	<code>id</code> , <code>task_id</code> , <code>provider</code> , <code>model</code> , <code>attempt_number</code> , <code>status</code> , <code>error_type</code> , <code>latency_ms</code> , <code>token_input</code> , <code>token_output</code> , <code>cost_estimate</code> , <code>raw_response</code> , <code>is_parallel</code> , <code>parallel_group</code> , <code>created_at</code> .
provider_health	<code>provider_id</code> , <code>status</code> , <code>failure_count</code> , <code>schema_failure_count</code> , <code>avg_latency_ms</code> , <code>last_success</code> , <code>last_failure</code> , <code>updated_at</code> — circuit breaker state.
memory_items	<code>id</code> , <code>user_id</code> , <code>layer</code> , <code>title</code> , <code>content</code> , <code>authority_level</code> , <code>created_at</code> , <code>updated_at</code> — Canon and friends.
attachments	Attachment metadata + <code>extraction_status</code> + per-file processing notes; linked to <code>task_id</code> once the task row is minted.

Together these tables are sufficient to reconstruct any task's full decision trace: what came in, what the gate said, what the Tri-State vector and signals were, which models were selected and what they returned, whether validation passed, what was repaired, and what was finally returned to the caller.

14. Threats the governance layer is designed to mitigate

Misuse of the LLM	Hard-block patterns in the input gate + ABORT amplitude on <code>illegal_instruction</code> / <code>unauthorized_action</code> / <code>unsafe_request</code> signals.
Silent unsafe output	Output validation has its own unsafe-output pattern set independent of the input gate.
Hallucination shipped as fact	<code>validation.completeness_pass</code> requires <code>recommended_next_action</code> ; confidence is attenuated by self-reported uncertainties and <code>failure_modes</code> ; HOLD is the safe default when nothing collapses

cleanly.

Provider outage / runaway cost

Circuit breaker excludes failed providers from routing; latency-aware EWMA promotes slow providers to DEGRADED; cost factors into

routing score.

Branding leakage

Repair engine rewrites self-references to BOS-OMEGA before the answer is persisted.

Prompt injection from attachments

Attachments are treated as contextual content blocks, not as instructions; vision bytes are gated by capability tag; transcription failures are honest, not fabricated.

Loss of audit trail

Audit writes never throw; the entire decision graph is reconstructable from the persisted tables; task_id is the join key everywhere.

Unauthorised configuration changes

Single-admin auth, signed cookies, type-the-title confirmation for canon deletes, encrypted provider keys at rest.

15. Design principles

- Default to HOLD. Whenever a rule does not resolve cleanly, the safe action is to ask for more, not to act.
- Make every policy decision auditable. Nothing important happens without a row in audit_logs and, where applicable, a structured row in tri_state_decisions or validation_results.
- Keep policy modules small and pure. Each governance file in artifacts/api-server/src/bos/ has a single responsibility and a clear input/output contract.
- Be honest about gaps. If transcription is unavailable, say so in attachment notes; if a multimodal model is missing the capability tag, do not silently drop images — refuse cleanly.
- Treat Canon as system law. Deleting Canon requires deliberate human confirmation; reading Canon happens in front of every model call.
- Push expensive checks to the edges. The input gate runs before any model is selected; validation and repair run after the model returns and before the answer is persisted.

16. v1.1 Hardening Addendum

This release adds twelve production controls in response to a hardened-spec review. The controls are layered so that when any one of them fails the safe default is always HOLD, never GO. Every denied or held task now also carries a structured plain-English explanation suitable for end-user presentation.

16.1 Hardened Tri-State Collapse

The collapse rule has been replaced with a sequence that defaults to HOLD unless GO is strongly justified. Implemented in artifacts/api-server/src/bos/triState.ts.

- 1. `hard_safety_abort` !' ABORT (hard veto, bypasses vector math).
- 2. `abort amplitude` "e 0.65 !' ABORT (down from 0.70).
- 3. `missing required inputs` !' HOLD.
- 4. `no provider available` !' HOLD.
- 5. `(go "e 0.75 "" validation_passed "" confidence "e requiredConfidence)` !' GO.
- 6. `otherwise` !' HOLD.
- `requiredConfidence` is 0.85 for legal / medical / financial / code / security tasks, 0.70 elsewhere.

16.2 Repair Engine — Defaults to HOLD

The repair engine no longer synthesises a `state=GO` `BosOutput` from malformed model output. Any structurally incomplete envelope collapses to HOLD with `repair_applied=true`, `failure_modes` contains `malformed_model_output`, and `recommended_next_action` asks the caller to retry with stricter structured-output enforcement. Implemented in `artifacts/api-server/src/bos/repairEngine.ts`.

16.3 Budget Governor

Per-mode ceilings on cost, model calls, fallbacks, repair attempts, parallel agents, synthesis retries, series depth, and validation retries. A breach forces HOLD with the `budget_exceeded` failure mode. Implemented in `artifacts/api-server/src/bos/budgets.ts` and applied in `pipeline.ts`.

normal	<code>max_models=1, max_fallbacks=2, max_repair_attempts=2, max_cost_usd=\$0.10.</code>
parallel	<code>max_models=4, max_fallbacks=2, max_repair_attempts=2, max_cost_usd=\$0.40.</code>
consensus	<code>max_models=5, max_fallbacks=2, max_repair_attempts=2, max_cost_usd=\$0.50.</code>
series_pass	<code>max_models=7, max_fallbacks=3, max_repair_attempts=2, max_cost_usd=\$0.50, max_series_depth=7.</code>
boil_the_ocean	<code>max_models=10, max_parallel_agents=10, max_synthesis_retries=2, max_repair_attempts=2, max_cost_usd=\$2.00.</code>

16.4 Loop Guards

The budget governor doubles as the loop guard:
`max_repair_attempts=2, max_fallbacks=3, max_series_depth=7, max_synthesis_retries=2, max_validation_retries=2`. Crossing any of them collapses the task to HOLD with a 'recursion / retry limit exceeded' explanation.

16.5 Deterministic Router Tie-Break

modelRouter.ts now sorts by composite score (desc), then capability_match, reliability_score, latency_score, cost_score, context_fit, and finally a lexicographic comparison on `provider\_id:model\_id`. Two identical configurations route the same way every time.

16.6 Attachment Injection Defense

All attachment-derived context is now wrapped in a `[UNTRUSTED ATTACHMENT CONTENT]` block that explicitly tells the model to treat the body as data, not as system / developer / user instruction. The loader scans extracted text for prompt-injection patterns ("ignore previous instructions", "system prompt", "developer message", "you are now", "override policy", line-starting `assistant:`, "act as", "disable safety") and:

- Adds a per-attachment note naming the offending file.
- Surfaces a separate ATTACHMENT_INJECTION_FLAGGED audit event.
- Keeps the content available, but with the warning preamble preserved through to the model.

16.7 Memory Layer Token Budgets and Relevance

memoryEngine.ts now enforces per-layer token budgets — canon=3000, patches=1000, continuity=1500, scratchpad=750 — and ranks items inside each layer by authority_level, then relevance to the current task input (keyword overlap), then recency, before greedy-filling the budget. Canon still wins as a layer, but it no longer floods context with off-topic items.

16.8 Audit Durability

auditEngine.ts retries the DB write up to three times with linear backoff. If all retries fail the event is appended to a durable JSONL queue under .local/audit-queue/pending.jsonl, a CRITICAL_AUDIT_FAILURE log line is emitted, and an in-process flag is set. When COMPLIANCE_MODE=true, the pipeline reads that flag and forces the task to HOLD per the failure-mode matrix.

16.9 Denial / HOLD Explanation Engine

Every BosOutput in HOLD or ABORT now carries three fields the UI can render verbatim:

why_decision_was_made	A plain-English governance explanation of the specific rule that fired.
safe_alternative	An allowed alternative path the user can pursue.
recommended_next_action	A concrete next step (existing field, now always populated).
	Causes covered: input_gate_abort, input_gate_hold_missing_info, tri_state_abort, tri_state_hold_no_provider, tri_state_hold_low_confidence, tri_state_hold_default, budget_exceeded, compliance_audit_failure.

16.10 Failure-Mode Matrix

Input gate failure	HOLD all tasks with explicit missing_info.
Tri-State engine failure	HOLD task — collapse defaults to HOLD.

Router finds no provider

Model timeout

Validation failure

Repair failure

Audit DB failure

Memory corruption

Attachment extraction fail

Budget exceeded

HOLD: provider unavailable (with denial explanation).

Fallback provider, then HOLD if exhausted.

Repair !' revalidate !' HOLD if still invalid.

HOLD; never GO. (repair_applied=true)

Retry !' durable queue !' CRITICAL_AUDIT_FAILURE !' HOLD in compliance mode.

Skip memory + alert (graceful degradation).

Continue with note; never fabricate.

HOLD with budget_exceeded failure mode.

16.11 RBAC — Acknowledged Scope

RBAC (Super Admin / Admin / Operator / Auditor / Read Only) was specified in the v1.1 hardening list but is intentionally NOT shipped in this release. The current authentication model is single-admin; a five-role design requires a users table, sign-up / invitation flow, per-route guards, and a session-shape migration. Treating RBAC as a separate, scoped task avoids rushing a partial implementation that would weaken the rest of the v1.1 controls. The denial explanation engine, audit durability, and budget governor described above all compose cleanly on top of any future RBAC layer.

16.12 Self-Check

- The repair engine state=GO default is FIXED. Malformed output now collapses to HOLD with repair_applied=true.
- Tri-State collapse defaults to HOLD; GO requires go"e0.75 and validation_passed and confidence"erequiredConfidence.
- Budget governor and loop guards are in the pipeline.
- Router tie-break is deterministic.
- Attachments are wrapped as untrusted data with injection patterns flagged.
- Memory layers enforce per-layer token budgets and rank by relevance.
- Audit writes are durable through DB retries plus an on-disk JSONL queue, with COMPLIANCE_MODE escalation to HOLD.
- Every denied or held task carries why_decision_was_made and safe_alternative.